

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

B. Tech CSE - Cloud Computing

COURSE NOTES

Containerization and Docker:

Concepts, Architecture, Life Cycle, Images, and Command Reference

Prepared for classroom instruction and self-study

Compiled by: Ajay Kumar Badhan, Assistant Professor, Dept. of CSE

Academic Year 2026

Table of Contents

1. Introduction to Containerization

1.1 What is Containerization?

Containerization is a lightweight form of virtualization in which an application, along with all its dependencies, libraries, and configuration files, is packaged together into a single unit called a container. A container shares the host machine's operating system kernel but runs in an isolated user space, so it behaves as if it has its own OS, file system, and network stack — without the overhead of running a full guest operating system.

The core idea is captured in the phrase "build once, run anywhere": a container that runs correctly on a developer's laptop will run identically on a testing server, a production cluster, or a cloud virtual machine, because the container carries its entire runtime environment with it.

1.2 Containers vs Virtual Machines

Virtual Machines (VMs) virtualize the hardware — each VM includes a full copy of a Guest Operating System, made possible by a hypervisor. Containers, in contrast, virtualize the operating system itself — all containers on a host share the same OS kernel, and only the application layer (binaries, libraries, code) differs between them. This makes containers dramatically smaller, faster to start, and more resource-efficient than VMs.

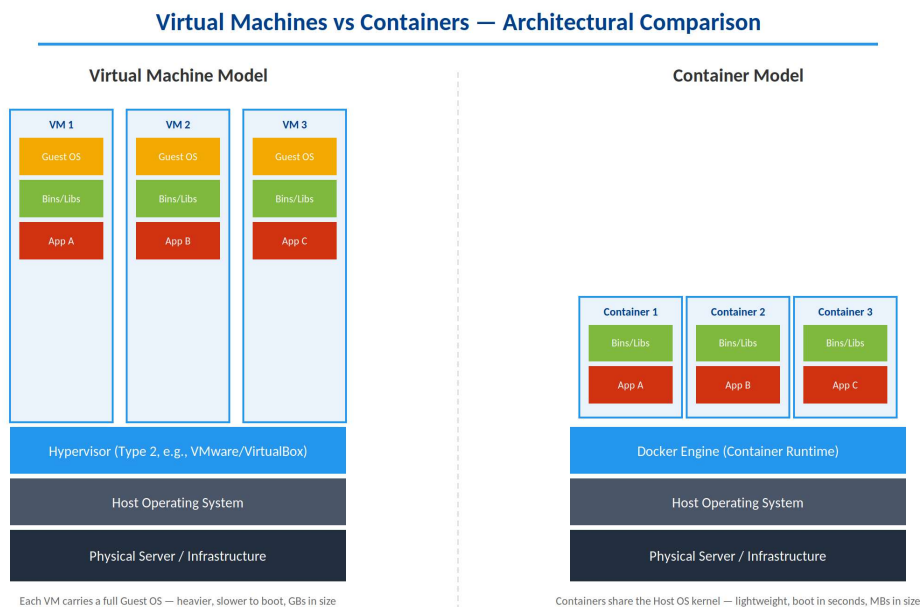


Fig 1: Containers virtualize the operating system, whereas VMs virtualize the underlying hardware

Fig 1: Containers virtualize the OS; Virtual Machines virtualize the hardware

1.3 Benefits of Containerization

- **Portability:** A container image runs the same way on any host with a compatible container runtime — laptop, on-prem server, or any cloud.
- **Lightweight:** Containers share the host kernel, so they start in milliseconds/seconds and are typically megabytes, not gigabytes, in size.
- **Isolation:** Each container has its own process space, file system view, and network interface, preventing conflicts between applications.

- Consistency: Eliminates the classic "it works on my machine" problem, since dev, test, and production environments are identical.
- Efficient resource utilization: Many more containers than VMs can run on the same hardware, since there is no per-instance guest OS overhead.
- Faster CI/CD: Containers integrate naturally with automated build, test, and deployment pipelines.
- Scalability: Containers can be started, stopped, and replicated very quickly, which is ideal for microservices and auto-scaling architectures.

1.4 Popular Container Platforms

- Docker — the most widely used containerization platform and the primary focus of this document.
- Podman — a daemon-less container engine, compatible with Docker commands, popular for rootless containers.
- containerd and CRI-O — lower-level container runtimes, often used internally by Kubernetes.
- LXC/LXD (Linux Containers) — an earlier, OS-level virtualization technology that influenced Docker's design.

While several platforms exist, Docker remains the de facto standard for learning containerization concepts, and the commands and workflows you learn on Docker transfer easily to other tools.

2. Docker Fundamentals

2.1 What is Docker?

Docker is an open-source platform that automates the deployment of applications inside lightweight, portable containers. Released in 2013, Docker popularized containerization by providing a simple command-line interface, a standard image format, and an ecosystem (Docker Hub, Docker Compose, Docker Swarm) that made containers accessible to mainstream developers, not just large infrastructure teams.

Docker allows a developer to define an application's environment once in a text file called a Dockerfile, build that definition into an Image, and then run any number of Containers from that image — each an isolated, running instance of the application.

2.2 Docker Hub

Docker Hub is the default public registry for Docker images — think of it as "GitHub for container images". It hosts millions of images, including:

- **Official Images:** Curated and maintained by Docker and the software vendors themselves (e.g., ubuntu, python, node, mysql, nginx) — safe, well-documented starting points.
- **Verified Publisher Images:** Published by verified organizations (e.g., Microsoft, Bitnami).
- **Community / User Images:** Public repositories pushed by individual developers, identified as username/image-name.

Key Docker Hub concepts every student should know:

- **Repository:** A collection of related images, usually different versions of the same application, identified by a name such as nginx or myusername/myapp.
- **Tag:** A label attached to a specific version of an image within a repository, e.g., python:3.11-slim or nginx:latest. If no tag is specified, Docker defaults to the latest tag.
- **Pull:** Downloading an image from a registry to your local machine (docker pull).
- **Push:** Uploading a locally built image to a registry so others can use it (docker push) — requires docker login first.
- **Private Repositories:** Docker Hub also supports private repositories for proprietary images, with access limited to authorized users/teams.

Besides Docker Hub, organizations often run their own private registries — Amazon ECR, Azure Container Registry (ACR), Google Artifact Registry, or a self-hosted Docker Registry/Harbor — for internal images that should not be public.

3. Docker Architecture

Docker uses a client-server architecture. The Docker Client talks to the Docker Daemon, which does the heavy lifting of building, running, and managing containers. The Client and Daemon can run on the same host (the common case for students) or the Client can connect to a remote Daemon over a network.

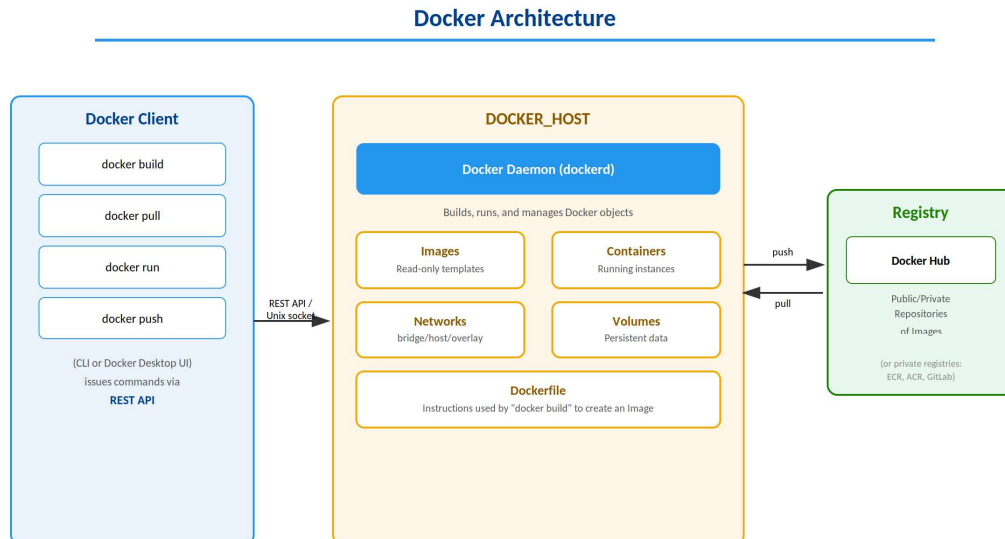


Fig 2: Docker's client-server architecture — Client, Daemon (dockerd), and Registry (Docker Hub)

Fig 2: Docker's client-server architecture

3.1 Docker Client

The component the user interacts with directly — either the docker command-line tool or a GUI such as Docker Desktop. Every command you type (docker run, docker build, docker pull ...) is translated into a REST API call sent to the Docker Daemon, typically over a Unix socket (Linux/Mac) or a named pipe (Windows).

3.2 Docker Daemon (dockerd)

The background service that does all the real work: building images, running and monitoring containers, managing networks and volumes, and communicating with registries. The daemon listens for API requests and manages Docker objects. A single daemon can also communicate with other daemons to manage a cluster of hosts (Docker Swarm).

3.3 Docker Objects

Images: Read-only templates containing the application code, runtime, libraries, and configuration required to run it. Images are built from a Dockerfile and are immutable — once built, an image does not change.

Containers: Runnable instances of an image. A container is the image plus a thin writable layer on top, plus runtime configuration (networking, environment variables, resource limits).

Networks: Docker's virtual networking layer (bridge, host, overlay, none drivers) that controls how containers communicate with each other and the outside world.

Volumes: The recommended mechanism for persisting data generated and used by containers, independent of the container's life cycle.

3.4 Registry

A registry stores and distributes Docker images. Docker Hub is the default public registry, but private registries (ECR, ACR, Harbor, GitLab Container Registry) are common in enterprises. `docker pull` retrieves an image from a registry; `docker push` publishes a locally built image to a registry.

4. Docker Container Life Cycle

A Docker container moves through a well-defined set of states from the moment it is created until it is deleted. Understanding these states and the exact command that triggers each transition is essential for debugging and for writing correct automation scripts.

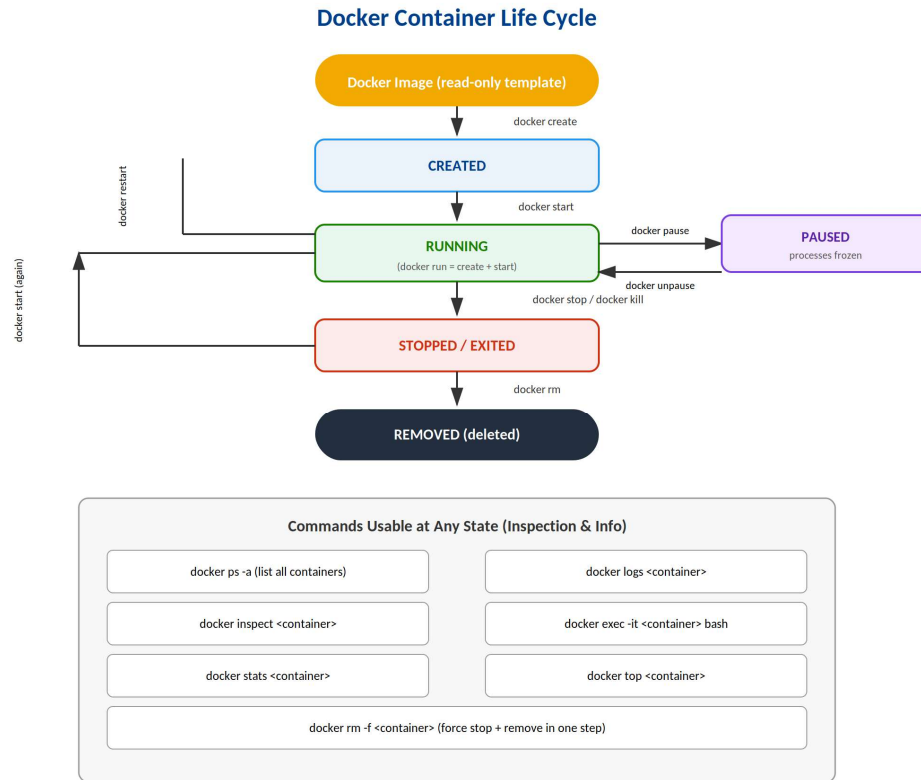


Fig 3: State transitions of a Docker container from Image to Removal

Fig 3: State transitions of a Docker container, from Image to Removal

4.1 Life Cycle States

- **Created:** The container's file system and configuration exist, but its main process has not started yet (produced by `docker create`).
- **Running:** The container's main process is executing (produced by `docker start` on a created container, or directly by `docker run` which combines `create + start`).
- **Paused:** All processes inside the container are frozen (suspended) using the host's cgroups freezer — useful for temporarily halting a container without losing its state (`docker pause/docker unpause`).
- **Stopped / Exited:** The main process has terminated, either gracefully (`docker stop`, which sends `SIGTERM` then `SIGKILL` after a grace period) or forcefully (`docker kill`, which sends `SIGKILL` immediately). The container's file system still exists and can be restarted.
- **Removed:** The container and its writable layer are permanently deleted (`docker rm`). Any data not stored in a volume is lost at this point.

4.2 Restart Policies

Rather than manually restarting crashed containers, Docker supports automated restart policies specified at `docker run` time using the `--restart` flag:

- `no` (default): Never automatically restart the container.
- `on-failure[:max-retries]`: Restart only if the container exits with a non-zero (error) status code.
- `always`: Always restart the container if it stops, including on daemon restart.
- `unless-stopped`: Like `always`, but does not restart if the container was explicitly stopped by the user before the daemon restarted.

5. Docker Images

A Docker image is an immutable, read-only template used to create containers. Images are built in layers, each instruction in a Dockerfile produces a new layer, and Docker's union file system stacks these layers to present what looks like a single, unified file system to the running container.

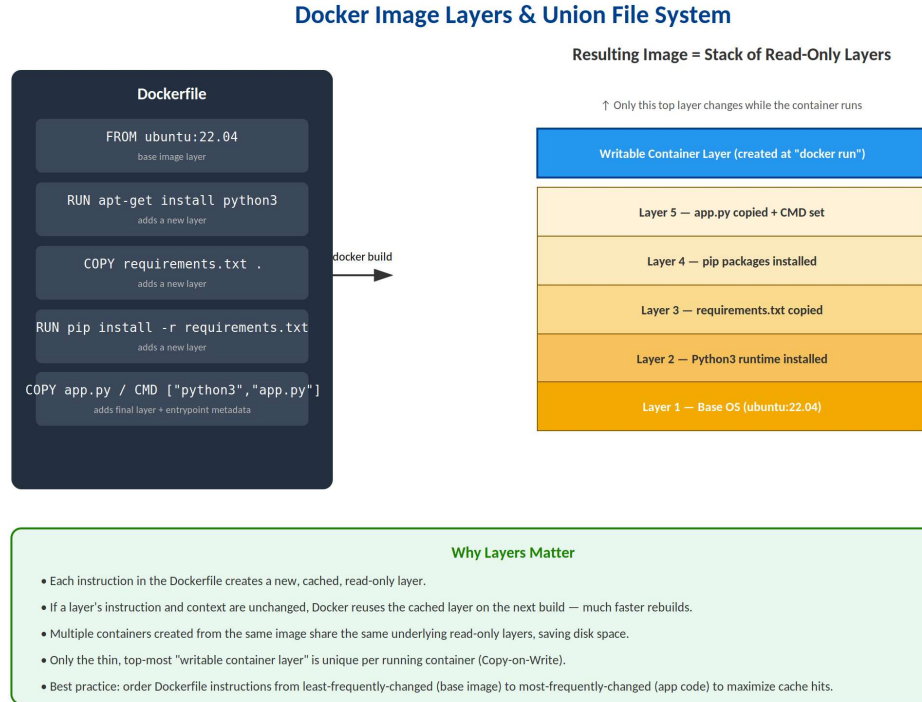


Fig 4: Each Dockerfile instruction produces a cached layer; containers add a thin writable layer on top

Fig 4: Each Dockerfile instruction produces a cached, reusable layer

5.1 The Dockerfile

A Dockerfile is a plain-text script containing a sequence of instructions that Docker executes, in order, to assemble an image. Below are the instructions every student must know:

Command Syntax	Description	Example
FROM <image>:<tag>	Specifies the base image to build from. Must be the first instruction (except ARG).	FROM python:3.11-slim
WORKDIR <path>	Sets the working directory for subsequent instructions and the container's default directory.	WORKDIR /app
COPY <src> <dest>	Copies files/folders from the build context (host) into the image.	COPY . /app
ADD <src> <dest>	Like COPY, but also supports remote URLs and automatic extraction of local tar archives.	ADD app.tar.gz /app

Command Syntax	Description	Example
RUN <command>	Executes a command at build time and commits the result as a new layer.	RUN pip install -r requirements.txt
ENV <key>=<value>	Sets an environment variable available at build time and in the running container.	ENV PORT=5000
EXPOSE <port>	Documents the port(s) the container listens on (does not actually publish the port).	EXPOSE 5000
CMD ["executable","arg1"]	Specifies the default command to run when a container starts (only one CMD, overridable at docker run).	CMD ["python3","app.py"]
ENTRYPOINT ["executable"]	Configures a container to run as an executable; CMD arguments are appended to it.	ENTRYPOINT ["python3"]
ARG <name>=<default>	Defines a build-time-only variable that can be passed via --build-arg.	ARG VERSION=1.0
VOLUME ["/data"]	Creates a mount point and marks it for holding externally mounted volumes.	VOLUME ["/var/lib/mysql"]
USER <username>	Sets the user (and optionally group) to run subsequent instructions and the container as.	USER appuser
LABEL <key>=<value>	Adds metadata to the image as key-value pairs.	LABEL maintainer="cse-dept"

5.2 Image Naming Convention

A fully qualified image name follows the pattern:

[registry-host]/[namespace]/repository[:tag].

For example, in docker.io/library/python:3.11-slim,

- docker.io is the registry host,
- library is the namespace for official images,
- Python is the repository, and
- 3.11-slim is the tag.

When the registry host is omitted, Docker Hub is assumed by default.

6. Docker Command Reference — Basic to Advanced

This section organizes the most important Docker commands from basic (installation, images, containers) to advanced (networks, volumes, Compose, Swarm, system maintenance) so you can use it as a quick-revision cheat sheet.

6.1 Installation and Setup (Basic)

Command Syntax	Description	Example
<code>docker --version</code>	Displays the installed Docker version.	<code>docker --version</code>
<code>docker info</code>	Displays system-wide information about the Docker installation.	<code>docker info</code>
<code>docker login</code>	Logs in to a Docker registry (Docker Hub by default).	<code>docker login -u myusername</code>
<code>docker logout</code>	Logs out from a Docker registry.	<code>docker logout</code>
<code>docker version</code>	Shows client and server (daemon) version details.	<code>docker version</code>

6.2 Working with Images (Basic)

Command Syntax	Description	Example
<code>docker pull <image>:<tag></code>	Downloads an image from a registry (Docker Hub by default).	<code>docker pull ubuntu:22.04</code>
<code>docker images / docker image ls</code>	Lists all images stored locally.	<code>docker images</code>
<code>docker build -t <name>:<tag> <path></code>	Builds an image from a Dockerfile located at <path>.	<code>docker build -t myapp:1.0 .</code>
<code>docker tag <src> <target></code>	Creates an additional tag/name for an existing local image.	<code>docker tag myapp:1.0 myuser/myapp:1.0</code>
<code>docker push <image>:<tag></code>	Uploads a local image to a registry.	<code>docker push myuser/myapp:1.0</code>
<code>docker rmi <image></code>	Removes one or more local images.	<code>docker rmi myapp:1.0</code>
<code>docker history <image></code>	Shows the layer history of an image.	<code>docker history python:3.11-slim</code>
<code>docker inspect <image></code>	Displays detailed low-level JSON information about an image.	<code>docker inspect python:3.11-slim</code>
<code>docker search <term></code>	Searches Docker Hub for images matching a keyword.	<code>docker search nginx</code>

6.3 Working with Containers (Basic to Intermediate)

Command Syntax	Description	Example
<code>docker run [options] <image> [cmd]</code>	Creates and starts a new container from an image.	<code>docker run -d -p 8080:80 --name web nginx</code>
<code>docker ps</code>	Lists currently running containers.	<code>docker ps</code>
<code>docker ps -a</code>	Lists all containers, including stopped ones.	<code>docker ps -a</code>
<code>docker start <container></code>	Starts an existing (stopped/created) container.	<code>docker start web</code>
<code>docker stop <container></code>	Gracefully stops a running container (SIGTERM, then SIGKILL).	<code>docker stop web</code>
<code>docker restart <container></code>	Restarts a container.	<code>docker restart web</code>
<code>docker pause / docker unpause</code>	Freezes/unfreezes all processes inside a container.	<code>docker pause web</code>
<code>docker kill <container></code>	Immediately stops a container (SIGKILL).	<code>docker kill web</code>
<code>docker rm <container></code>	Removes a stopped container.	<code>docker rm web</code>
<code>docker rm -f <container></code>	Force-stops and removes a container in one step.	<code>docker rm -f web</code>
<code>docker logs [-f] <container></code>	Shows (and optionally follows, -f) a container's console output.	<code>docker logs -f web</code>
<code>docker exec -it <container> <cmd></code>	Runs an additional command inside a running container interactively.	<code>docker exec -it web bash</code>
<code>docker attach <container></code>	Attaches your terminal to a running container's main process.	<code>docker attach web</code>
<code>docker cp <src> <container>:<dest></code>	Copies files/folders between the host and a container.	<code>docker cp ./index.html web:/usr/share/nginx/html/</code>
<code>docker top <container></code>	Lists the processes running inside a container.	<code>docker top web</code>
<code>docker stats [container]</code>	Shows live CPU, memory, and network usage statistics.	<code>docker stats web</code>
<code>docker rename <old> <new></code>	Renames a container.	<code>docker rename web webserver</code>
<code>docker diff <container></code>	Shows file system changes made inside a container since it started.	<code>docker diff web</code>

6.4 Common Docker Run Options (Frequently Tested)

Command Syntax	Description	Example
<code>-d, --detach</code>	Runs the container in the background and prints the container ID.	<code>docker run -d nginx</code>
<code>-p host:container</code>	Publishes a container's port to the host.	<code>docker run -p 8080:80 nginx</code>
<code>-v host_path:container_path</code>	Mounts a host directory or named volume into the container.	<code>docker run -v mydata:/app/data myapp</code>
<code>--name <name></code>	Assigns a custom, human-readable name to the container.	<code>docker run --name web nginx</code>
<code>-e KEY=VALUE</code>	Sets an environment variable inside the container.	<code>docker run -e ENV=production myapp</code>
<code>--rm</code>	Automatically removes the container when it exits.	<code>docker run --rm alpine echo hi</code>
<code>-it</code>	Combines <code>-i</code> (interactive) and <code>-t</code> (allocate a pseudo-TTY) for interactive shells.	<code>docker run -it ubuntu bash</code>
<code>--network <name></code>	Connects the container to a specific Docker network.	<code>docker run --network mynet myapp</code>
<code>--memory / --cpus</code>	Limits the memory / CPU resources available to the container.	<code>docker run --memory=512m --cpus=1 myapp</code>
<code>--restart <policy></code>	Sets the container's automatic restart policy.	<code>docker run --restart unless-stopped myapp</code>

6.5 Networking (Intermediate to Advanced)

Command Syntax	Description	Example
<code>docker network ls</code>	Lists all Docker networks.	<code>docker network ls</code>
<code>docker network create <name></code>	Creates a new user-defined bridge network.	<code>docker network create mynet</code>
<code>docker network inspect <name></code>	Shows detailed configuration of a network.	<code>docker network inspect mynet</code>
<code>docker network connect <net> <container></code>	Connects a running container to an additional network.	<code>docker network connect mynet web</code>
<code>docker network disconnect <net> <container></code>	Disconnects a container from a network.	<code>docker network disconnect mynet web</code>
<code>docker network rm <name></code>	Removes one or more user-defined networks.	<code>docker network rm mynet</code>

6.6 Volumes and Persistence (Intermediate to Advanced)

Command Syntax	Description	Example
<code>docker volume create <name></code>	Creates a named volume managed by Docker.	<code>docker volume create dbdata</code>
<code>docker volume ls</code>	Lists all Docker volumes.	<code>docker volume ls</code>
<code>docker volume inspect <name></code>	Shows detailed information about a volume, including its host path.	<code>docker volume inspect dbdata</code>
<code>docker volume rm <name></code>	Removes one or more volumes (must be unused).	<code>docker volume rm dbdata</code>
<code>docker volume prune</code>	Removes all unused local volumes.	<code>docker volume prune</code>

6.7 Docker Compose — Multi-Container Applications (Advanced)

Docker Compose lets you define and run multi-container applications using a single declarative YAML file (`docker-compose.yml`), instead of chaining together many long `docker run` commands.

```
# docker-compose.yml
version: "3.9"
services:
  web:
    build: .
    ports:
      - "5000:5000"
    depends_on:
      - db
  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: example
    volumes:
      - dbdata:/var/lib/mysql
volumes:
  dbdata:
```

Command Syntax	Description	Example
<code>docker compose up [-d]</code>	Builds (if needed) and starts all services defined in <code>docker-compose.yml</code> .	<code>docker compose up -d</code>
<code>docker compose down</code>	Stops and removes containers, networks created by up.	<code>docker compose down</code>
<code>docker compose ps</code>	Lists containers belonging to the Compose project.	<code>docker compose ps</code>
<code>docker compose logs [-f]</code>	Shows logs from all services.	<code>docker compose logs -f</code>
<code>docker compose build</code>	Builds/rebuilds service images without starting them.	<code>docker compose build</code>

Command Syntax	Description	Example
<code>docker compose restart</code>	Restarts all services.	<code>docker compose restart</code>
<code>docker compose exec <service> <cmd></code>	Runs a command inside a running service container.	<code>docker compose exec web bash</code>

6.8 Docker Swarm — Orchestration Basics (Advanced)

Docker Swarm is Docker's native clustering and orchestration tool, allowing a group of Docker hosts to be managed as a single virtual system, distributing containers ('services') across multiple nodes for high availability and scaling.

Command Syntax	Description	Example
<code>docker swarm init</code>	Initializes the current node as a Swarm manager.	<code>docker swarm init</code>
<code>docker swarm join --token <token> <ip></code>	Joins a node to an existing Swarm as a worker/manager.	<code>docker swarm join --token SWMTKN-... 192.168.1.10:2377</code>
<code>docker node ls</code>	Lists all nodes in the Swarm (manager only).	<code>docker node ls</code>
<code>docker service create --name <name> <image></code>	Creates a new replicated service in the Swarm.	<code>docker service create --name web --replicas 3 -p 80:80 nginx</code>
<code>docker service ls</code>	Lists all services running in the Swarm.	<code>docker service ls</code>
<code>docker service scale <service>=<n></code>	Scales a service to n replicas.	<code>docker service scale web=5</code>
<code>docker service rm <name></code>	Removes a service from the Swarm.	<code>docker service rm web</code>
<code>docker stack deploy -c <file> <name></code>	Deploys a multi-service stack from a Compose file to the Swarm.	<code>docker stack deploy -c docker-compose.yml mystack</code>

6.9 System Maintenance and Cleanup (Advanced)

Command Syntax	Description	Example
<code>docker system df</code>	Shows disk space used by images, containers, and volumes.	<code>docker system df</code>
<code>docker system prune</code>	Removes all stopped containers, unused networks, dangling images, and build cache.	<code>docker system prune -a</code>
<code>docker container prune</code>	Removes all stopped containers.	<code>docker container prune</code>
<code>docker image prune</code>	Removes dangling (untagged) images.	<code>docker image prune -a</code>
<code>docker builder prune</code>	Clears the Docker build cache.	<code>docker builder prune</code>

Command Syntax	Description	Example
docker events	Streams real-time events from the Docker daemon.	docker events
docker save -o <file>.tar <image>	Exports an image to a tar archive for offline transfer.	docker save -o myapp.tar myapp:1.0
docker load -i <file>.tar	Loads an image from a tar archive created by docker save.	docker load -i myapp.tar
docker export -o <file>.tar <container>	Exports a container's file system (not layers/history) as a tar archive.	docker export -o web.tar web

7. Practical Hands-On: Containerizing a Simple Python Web App

This walkthrough takes you from a plain Python script to a fully containerized, runnable application, using the concepts covered in this document. Follow along on your own machine with Docker Desktop or Docker Engine installed.

Step 1 — Create the application files

Create a project folder with two files: `app.py` (a minimal Flask web server) and `requirements.txt` (its one dependency).

```
# app.py
from flask import Flask
app = Flask(__name__)

@app.route("/")
def hello():
    return "Hello from inside a Docker container!"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)

# requirements.txt
flask==3.0.0
```

Step 2 — Write the Dockerfile

```
# Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python3", "app.py"]
```

Step 3 — Build the image

```
docker build -t flask-demo:1.0 .
```

Docker reads the Dockerfile top to bottom, creating one cached layer per instruction (Section 5). The `-t` flag tags the resulting image as `flask-demo:1.0`; the trailing `.` tells Docker to use the current directory as the build context.

Step 4 — Run a container from the image

```
docker run -d -p 5000:5000 --name flask-container flask-demo:1.0
```

This creates and starts a container in detached mode (`-d`), maps host port 5000 to container port 5000 (`-p 5000:5000`), and names the container `flask-container`. Open a browser (or use `curl`) at `http://localhost:5000` — you should see the greeting message.

Step 5: Inspect, monitor, and manage the running container

docker ps	# confirm the container is running
docker logs flask-container	# view Flask's console output
docker exec -it flask-container bash	# open a shell inside the container
docker stats flask-container	# view live CPU/memory usage

Step 6: Stop, restart, and clean up

docker stop flask-container	# STOPPED state
docker start flask-container	# back to RUNNING
docker rm -f flask-container	# force stop + REMOVED
docker rmi flask-demo:1.0	# remove the image itself

Step 7: (Optional) Share it on Docker Hub

docker login
docker tag flask-demo:1.0 <your-dockerhub-username>/flask-demo:1.0
docker push <your-dockerhub-username>/flask-demo:1.0

Anyone with Docker installed can now run your application anywhere in the world with a single command:

```
docker run -p 5000:5000 <your-dockerhub-username>/flask-demo:1.0
```

This is the essence of containerization's portability promise.

8. Exercise Questions

8.1 Programming Exercises

- Write a Dockerfile that containerizes a simple "Hello, World" program written in a language of your choice (Python/Node.js/Java). Build the image and run it, printing the output to the console.
- Extend the Flask application from Section 7 to include a second route, /about, that returns your name and roll number. Rebuild the image with a new tag (e.g., flask-demo:2.0) and run both versions (1.0 and 2.0) simultaneously on different host ports.
- Write a Dockerfile for a Node.js application that uses a multi-stage build — one stage to install dependencies and build the app, and a second, smaller final stage that only copies the built artifacts. Compare the final image size against a single-stage build using Docker images.
- Create a docker-compose.yml file that runs a simple web application container alongside a Redis container, with the web service depending on Redis. Verify both containers can communicate using docker compose exec.
- Write a shell script that automates the following: build an image, run a container from it, wait 10 seconds, print the container's logs, then stop and remove the container. Use appropriate Docker commands from Section 6.
- Demonstrate the difference between docker stop and docker kill by writing a small program that traps and logs the SIGTERM signal, then observing the different behavior under each command.
- Create two custom Docker networks and demonstrate that containers on the same network can ping each other by container name, while containers on different networks cannot, using docker network create and docker network connect.

8.2 Short Answer Questions

- Define containerization and list any three benefits over traditional virtual machines.
- What is the difference between a Docker image and a Docker container?
- What is Docker Hub, and what is the difference between an official image and a community image?
- List the three main components of Docker's client-server architecture.
- Differentiate between docker stop and docker kill.
- What is the purpose of the EXPOSE instruction in a Dockerfile? Does it actually publish a port to the host?
- What is a Docker volume, and why is it preferred over storing data directly inside a container's writable layer?
- What does the -d flag do in the Docker run command?
- Name any two restart policies supported by Docker and explain one use case for each.
- What is the difference between COPY and ADD instructions in a Dockerfile?

8.3 Long Answer Questions

- Explain the complete Docker container life cycle with a neat diagram, describing the exact command used at each state transition.
- Describe Docker's layered image architecture and union file system. Explain, with an example Dockerfile, how Docker's build cache improves rebuild performance, and how you would order instructions to maximize cache reuse.
- Compare containerization and virtualization in detail, covering architecture, resource overhead, startup time, isolation level, and typical use cases. Support your answer with a diagram.
- Explain Docker's client-server architecture in detail, describing the role of the Docker Client, Docker Daemon, and Registry, and how they communicate with each other.
- Discuss the role of Docker Compose in managing multi-container applications. Illustrate with a docker-compose.yml example containing at least two dependent services, explaining each key used.
- What is Docker Swarm? Explain how it differs from running standalone containers, and describe the process of creating, scaling, and removing a replicated service.
- Design a containerized three-tier web application (front-end, back-end API, and database), explaining which Docker objects (images, containers, networks, volumes) you would use for each tier and why.

9. Summary

- Containerization packages an application with all its dependencies into a portable, isolated unit that shares the host OS kernel, unlike VMs which virtualize entire hardware stacks.
- Docker is the leading containerization platform, built around Images (immutable templates), Containers (running instances), and a client-server architecture (Client, Daemon, Registry).
- Docker Hub is the default public registry for storing and sharing images through repositories and tags, and supports both public and private repositories.
- A container moves through well-defined life cycle states — Created, Running, Paused, Stopped, and Removed — each triggered by a specific Docker command.
- Docker images are built in cached, reusable layers defined by Dockerfile instructions, with a thin writable layer added per running container.
- Docker's command set scales from simple image/container management to advanced orchestration with Docker Compose and Docker Swarm, supporting everything from a single hands-on lab to production-grade, multi-host deployments.

10. References & Further Reading

- Docker Official Documentation — <https://docs.docker.com>
- Docker Hub — <https://hub.docker.com>
- Dockerfile Reference — <https://docs.docker.com/engine/reference/builder/>
- Docker Compose Documentation — <https://docs.docker.com/compose/>
- Docker Swarm Documentation — <https://docs.docker.com/engine/swarm/>

Note to students: Docker's CLI evolves over time (e.g., docker-compose the standalone tool has been superseded by docker compose as a built-in subcommand). Always verify command syntax against the current official documentation before an exam, interview, or project submission.